

CS360 Introduction to Database

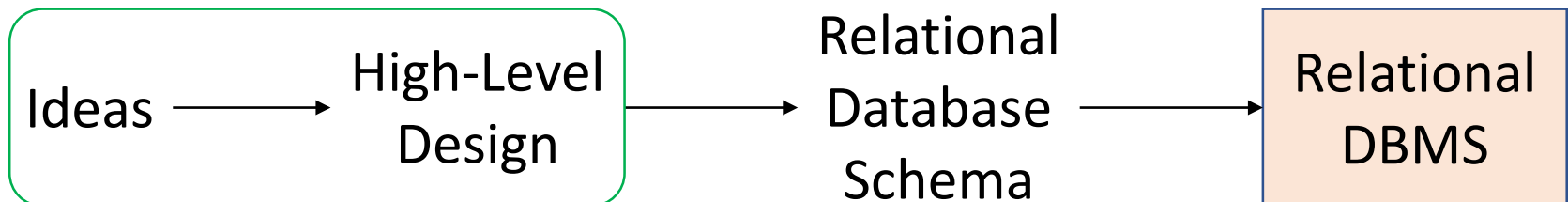
High-Level Database Models

Min-Soo Kim

School of Computing, KAIST

Process whereby a new DB is created

- Design phase
 - what information will be stored
 - how information elements will be related to one another
 - what constraints may be assumed
 - e.g., keys or referential integrity
- Notations in which design is expressed
 - entity-relationship diagram (the first, and oldest, method)
 - Unified Modeling Language (UML)
 - Object Description Language (ODL)



ER model

Entity/Relationship (ER) model

- Structure of data is represented graphically
 - i.e., entity-relationship diagram
- Three principal element types
 - entity set
 - relationship set
 - attribute
- **Entity set** (*usually converted to a relation in relational DB*)
 - **entity**: abstract object of some sort ($\approx$ *object* in OOP)
 - e.g., each movie, each studio, ...
 - **entity set**: a collection of similar entities ($\approx$ *class of objects*)
 - e.g., the set of all movies (Movies), the set of all studios (Studios), ...

- Relationship set

- relationship: connection among two or more entities

- e.g., Harrison Ford (*an* entity) acts in Star Wars (another entity)

- binary relationship (between two entities)

- ternary relationship (among three entities)

- multiway relationship (among multiple entities)

- relationship set: a collection of similar relationships

- e.g., connections between *Stars* entity set and *Movies* entity set

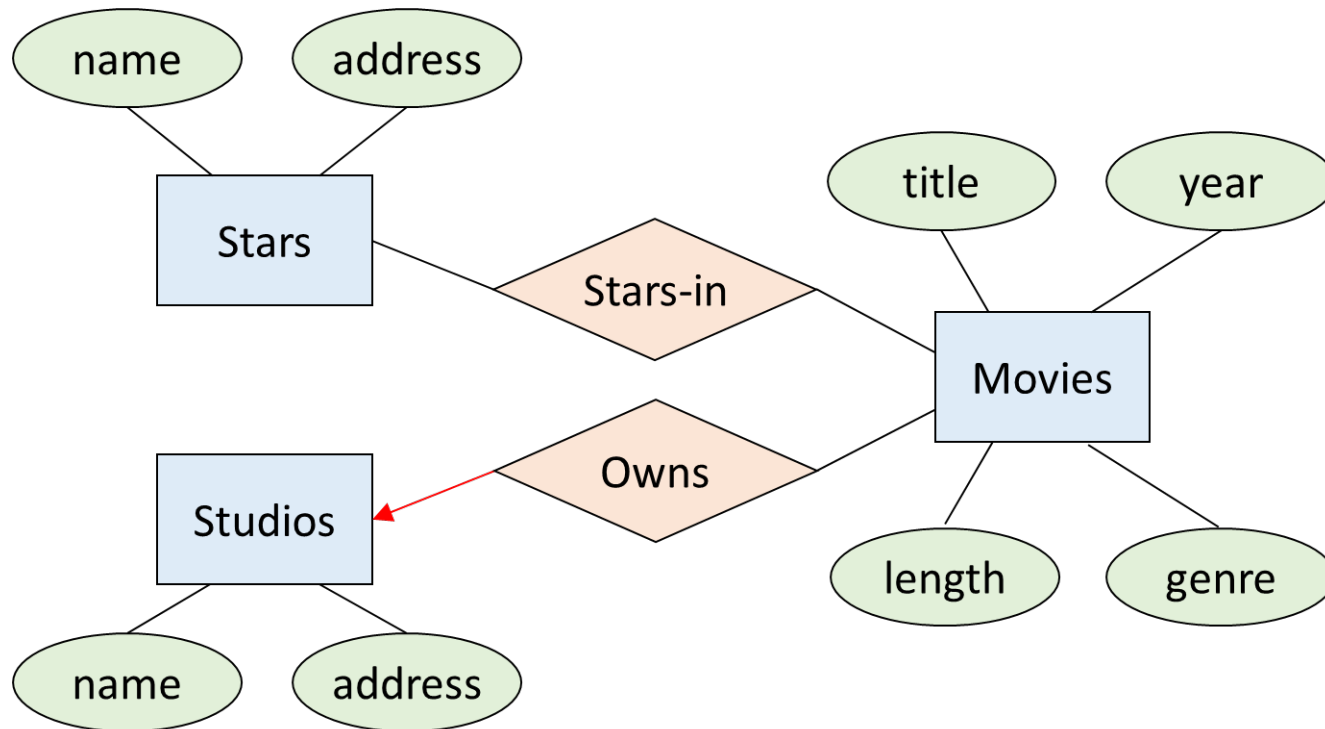
- Attribute (*usually converted to a column in relational DB*)

- entity sets or relationship sets have associated attributes

- e.g., *title* and *length* for the entity set *Movies*

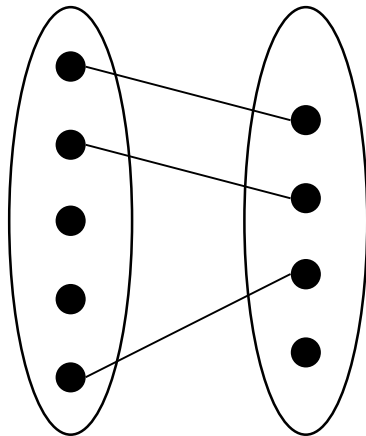
Entity-relationship (ER) diagrams

- Graph representing entity sets, relationship sets, attributes
 - entity set: rectangle
 - relationship set: diamond
 - attribute: oval

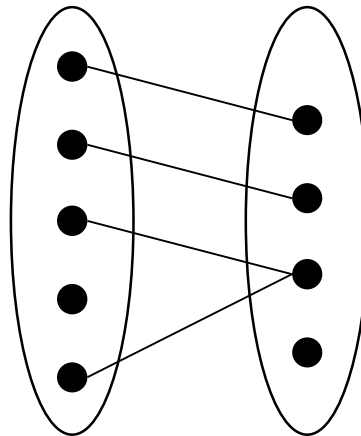


each movie is owned by at most one studio

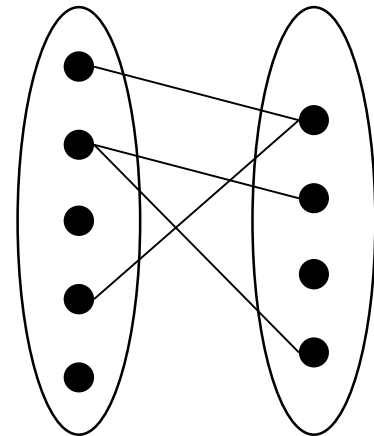
Multiplicity of binary ER relationship set



one-one

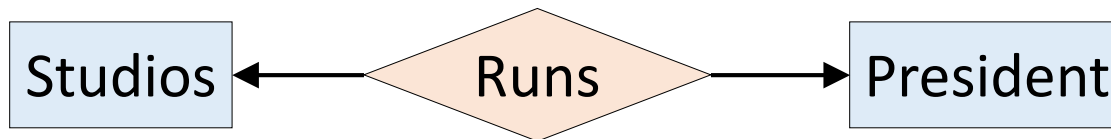


many-one



many-many

e.g.,

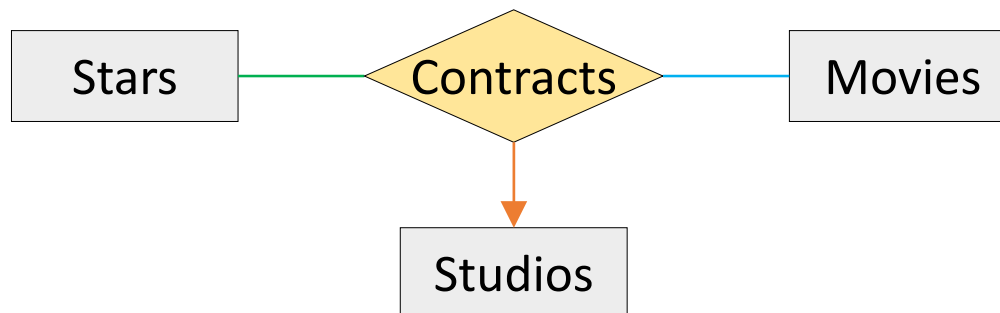


one-one relationship set

- the arrow means “**at most one**” and not “exactly one”
- a studio might not have a president at some particular time

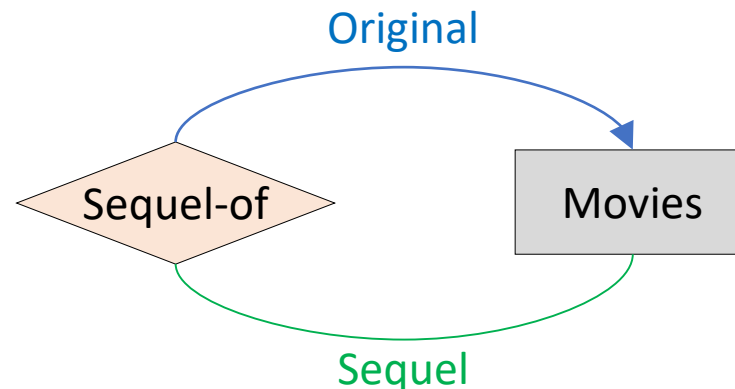
Multiway relationship set

- Ternary or multiway relationship sets are not very common
 - but, sometimes necessary to reflect the true state of affairs
- e.g., *Contracts* (ternary relationship set)
 - a studio has contracted with a particular star to act in a particular movie
 - each relationship can be described by a triple of three entities
 - at most one studio for a particular star and movie
 - but, a studio may contract with several stars for a movie
 - a star may contract with one studio for more than one movie



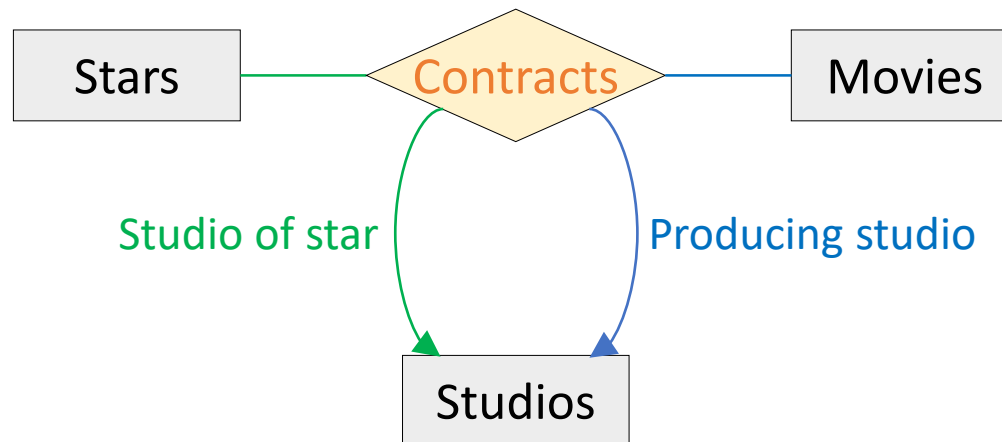
Roles in relationship set

- One entity set may appear two or more times in a single relationship set
 - draw as many lines from the relationship set to the entity set as the entity set appears in the relationship set
 - each line represents a different **role** that the entity set plays in the relationship set
- e.g., relationships between two movies
 - a movie may have many sequels
 - but, for each sequel, there is only one original movie

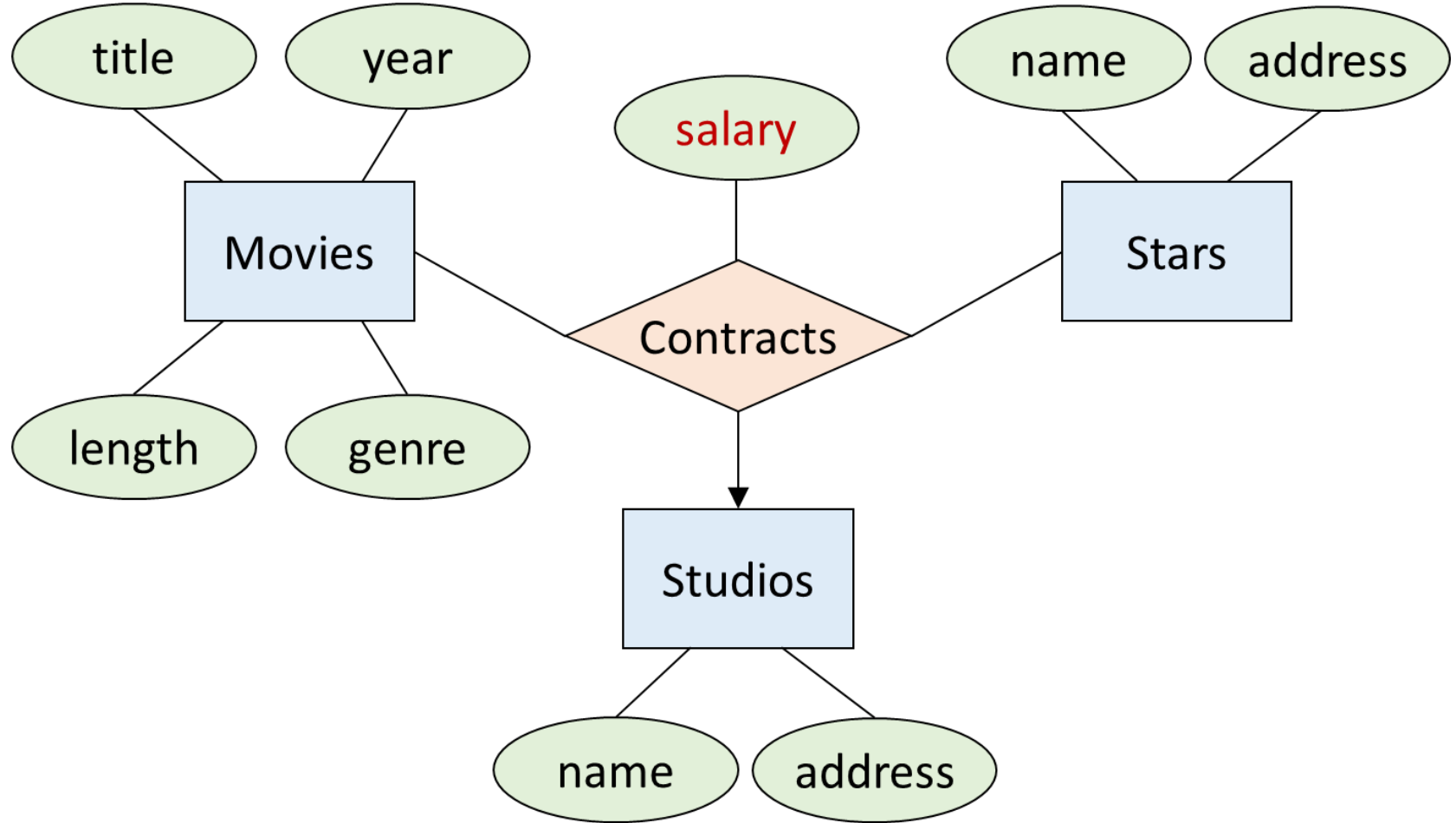


Example: more complex version of *Contracts*

- One contract involves two studios
 - one studio, having a certain star under contract, may further **contract** with a second studio to allow that star to act in a particular movie
- 4-tuple representation: (studio1, studio2, star, movie)

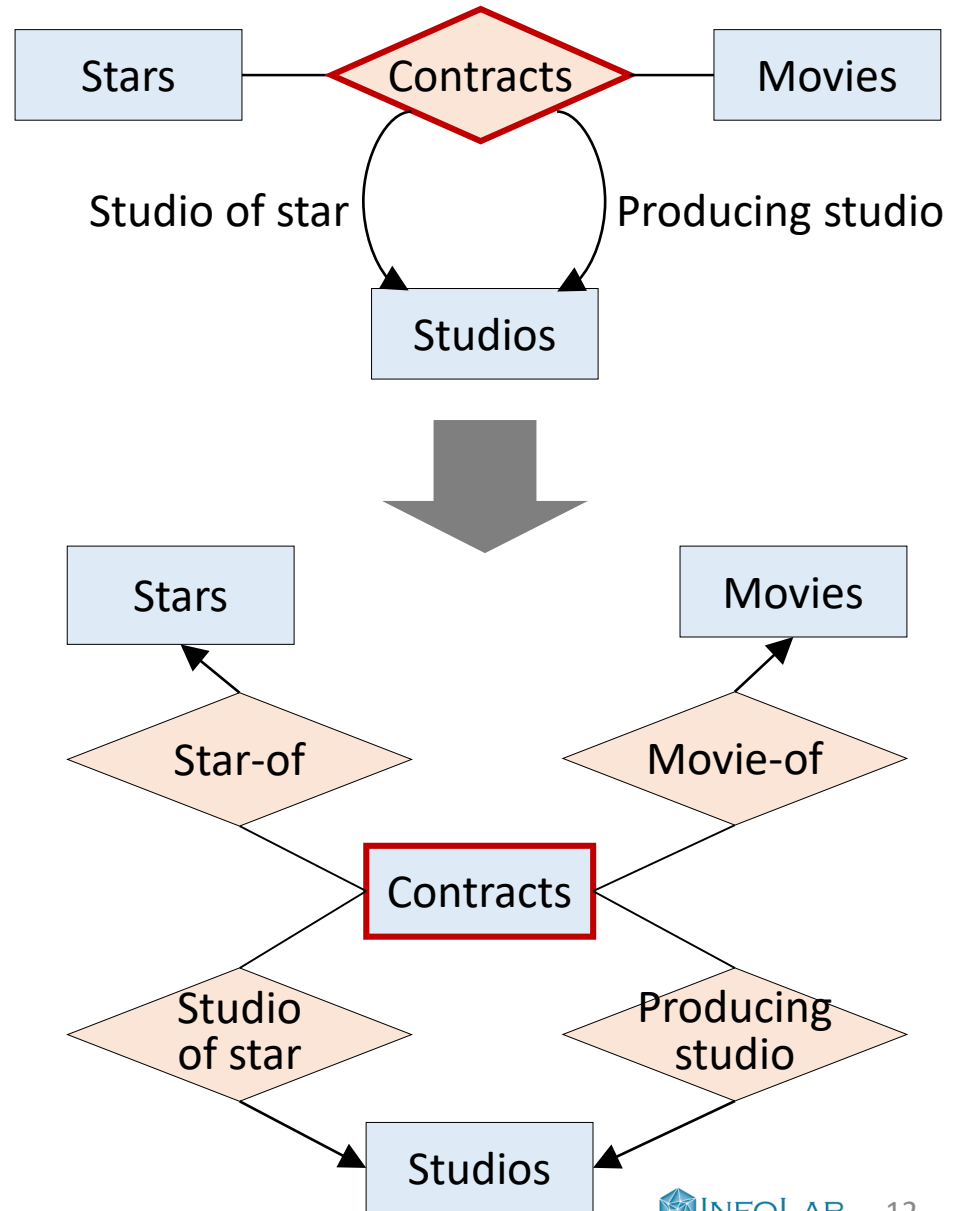


Attributes on entity sets and relationship sets



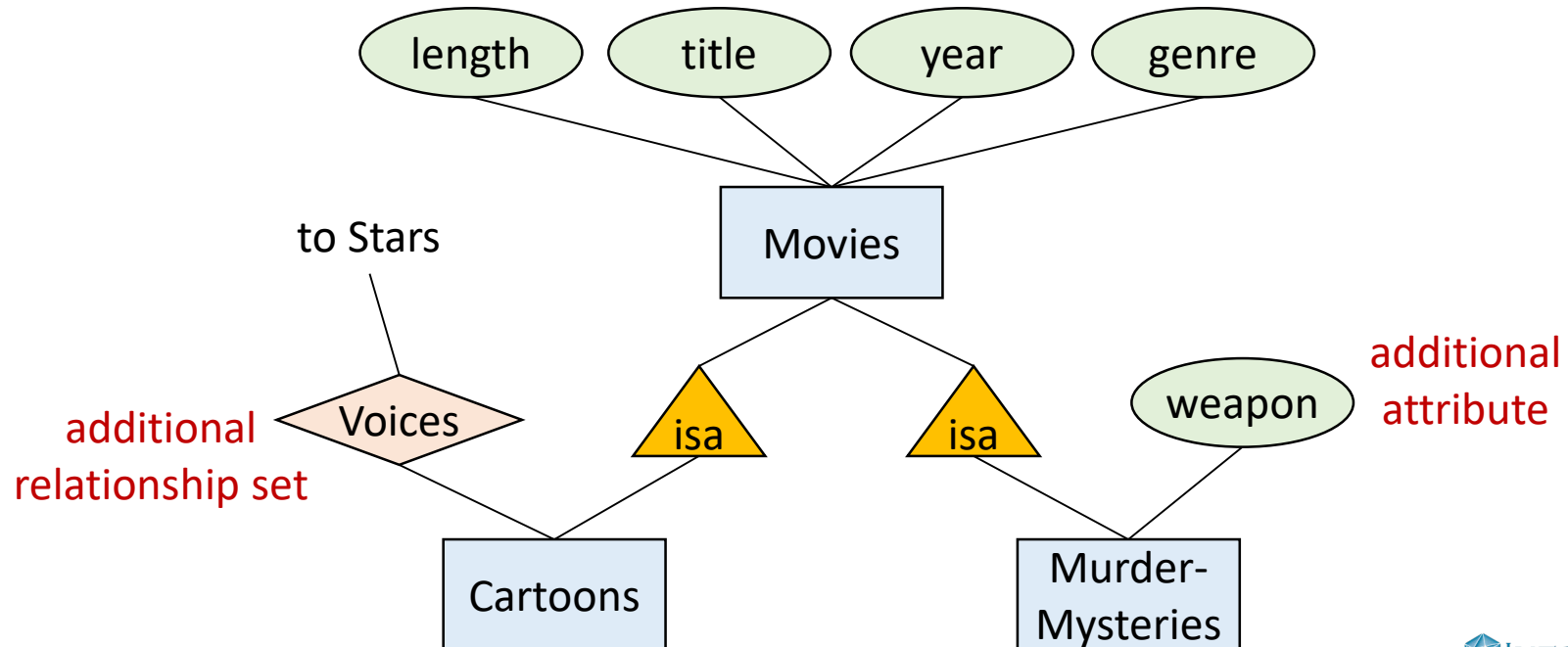
Converting multiway relationships to binary

- It may be useful to convert multiway to binary
 - UML and ODL limit relationships to be binary
 - introduce a new entity set (**connecting** entity set)
 - introduce many-one binary relationship sets from the connecting entity set to each of other entity sets
 - e.g., *Contracts* has a 4-tuple (studio1, studio2, star, movie)



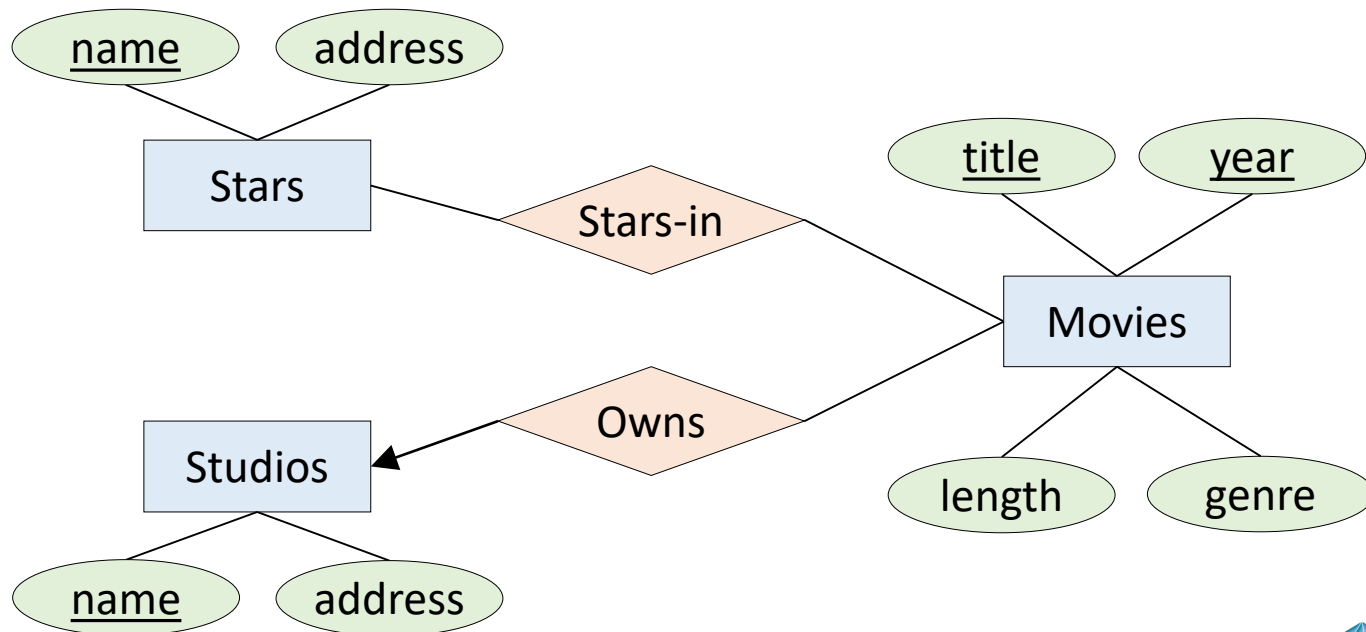
Subclasses in the ER model

- We can define certain special-case entity sets (subclasses)
- We connect an entity set to its subclasses using “isa”
 - isa : a special kind of relationship set (denoted as triangle)
 - isa relationship set is one-one (although not draw two arrows)
- e.g., special movie types



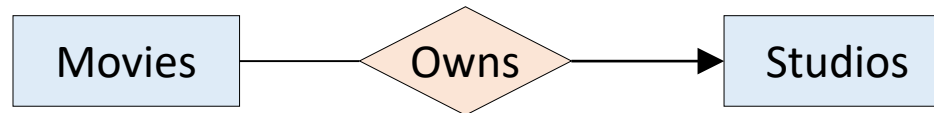
Keys in the ER model

- Every entity set **must have a key**
 - except subclasses and weak entity sets
- There can be more than one possible key for an entity set
 - it is customary to pick one key as the **primary key** (**underlined**)
- In isa-hierarchy, **root must have all the attributes for a key**



Referential integrity constraint

- Value appearing in one context **MUST** also appear in another
- e.g., many-one relationship set *Owns*
 - no movie can be owned by more than one studio
 - not say that a movie must surely be owned by a studio

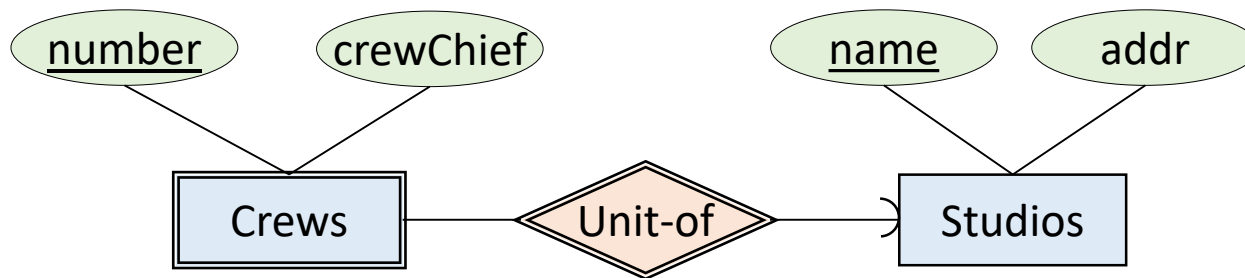


- Rounded arrow: N:1 / 1:1 + referential integrity constraint
 - each movie **MUST** be owned by one studio in *Studios* entity set
 - each president runs a studio that exists in *Studios* entity set
 - a studio would continue to exist, even if a president were fired



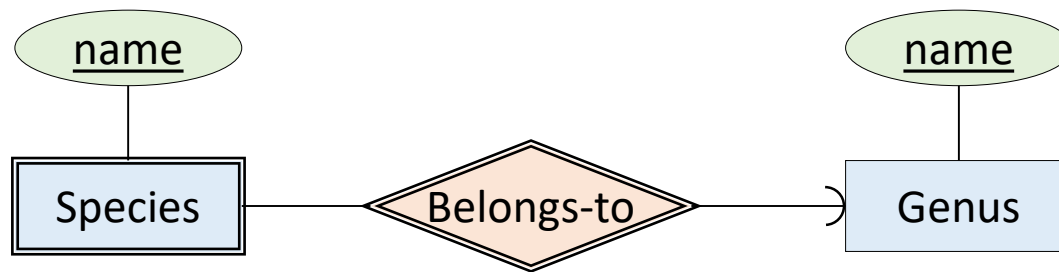
Weak entity sets E

- E 's key is composed of some or all attributes of another entity set F
 - if the entities of E are subordinate to the entities of F
 - the entities of E are not unique until taking into account the entity of F
 - e.g., the *number* of *Crews* is not a key for crews
 - rather we need both the *name* of *Studios* and the *number* of *Crews*



- double-rectangle: a weak entity
- double-diamond: many-one relationship set for the weak entity

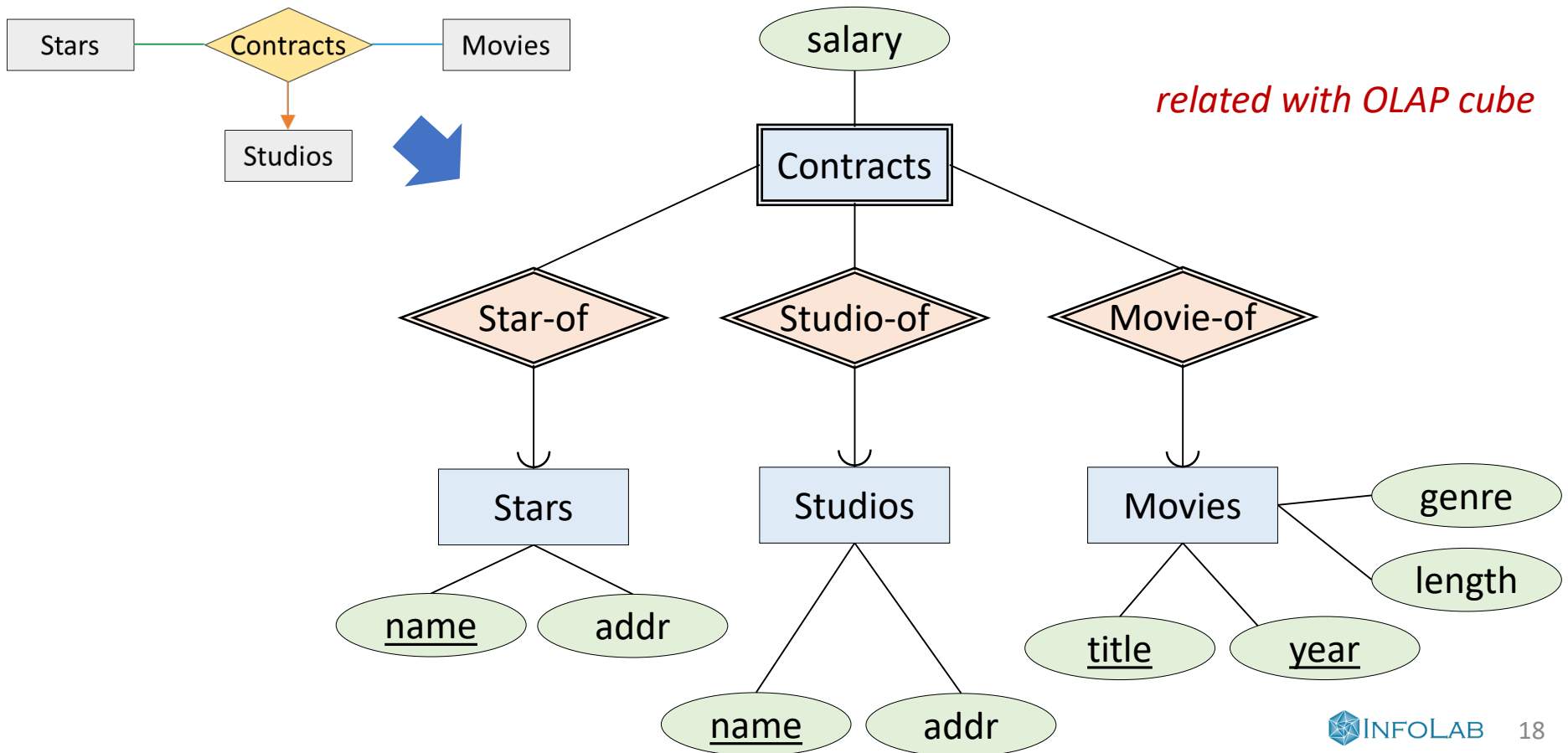
- e.g., a species is designated by its genus and species names
 - Homo : genus name of humans
 - sapiens : species name of humans
 - species names, by themselves, are not unique



- Note: many-one relationship set from weak entity set to supporting entity set must be rounded arrow

Connecting entity sets are also weak entity sets

- Introduced by converting multiway relationships to binary
 - often have no attributes of their own
 - their key is formed from the key of the entity sets they connect

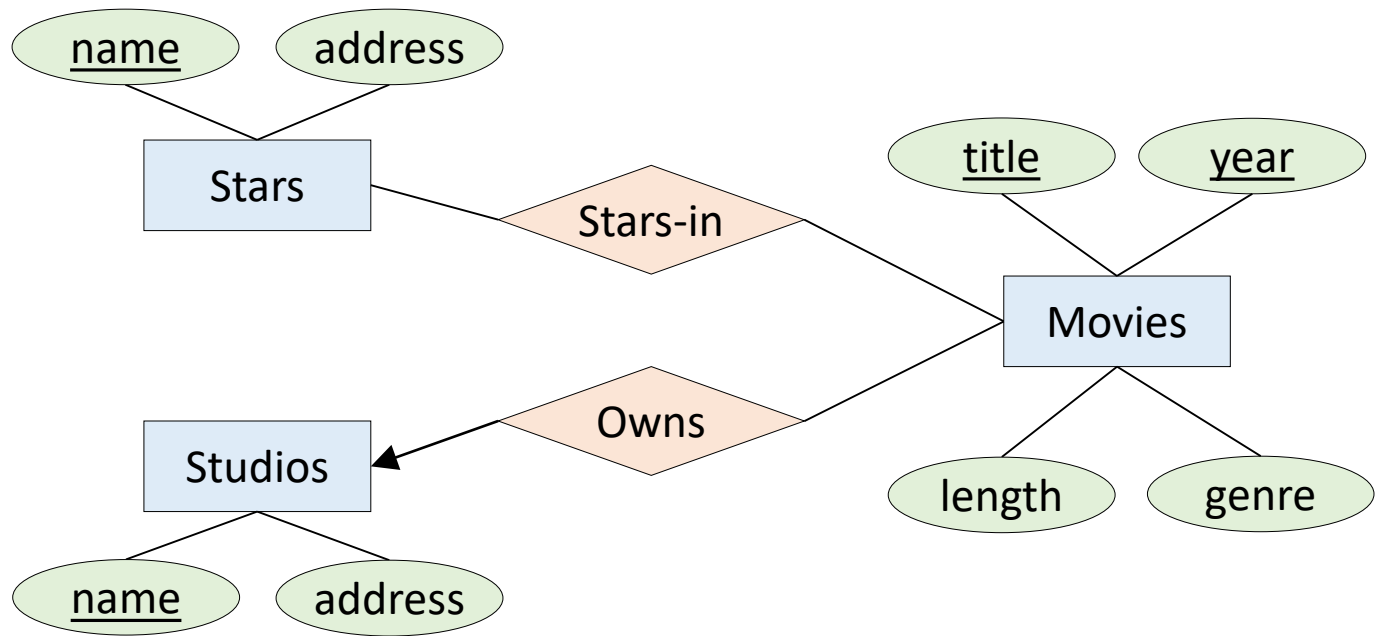


From ER diagram to relational database schema

From ER diagram to relational database schema

- Basic two rules for converting
 - turn each entity set into a relation with the same set of attributes
 - replace a relationship set by a relation whose attributes are
 - the attributes of the relationship set
 - the key attributes of the entity sets connected by the relationship set
 - note: we must rename the attributes to avoid name duplication
- Issues
 - ① we may need to combine two relations, especially connected by a many-one relationship set
 - ② weak entity sets cannot be turned straightforwardly into relations
 - ③ “isa” relationship sets and subclasses require careful treatment

Example



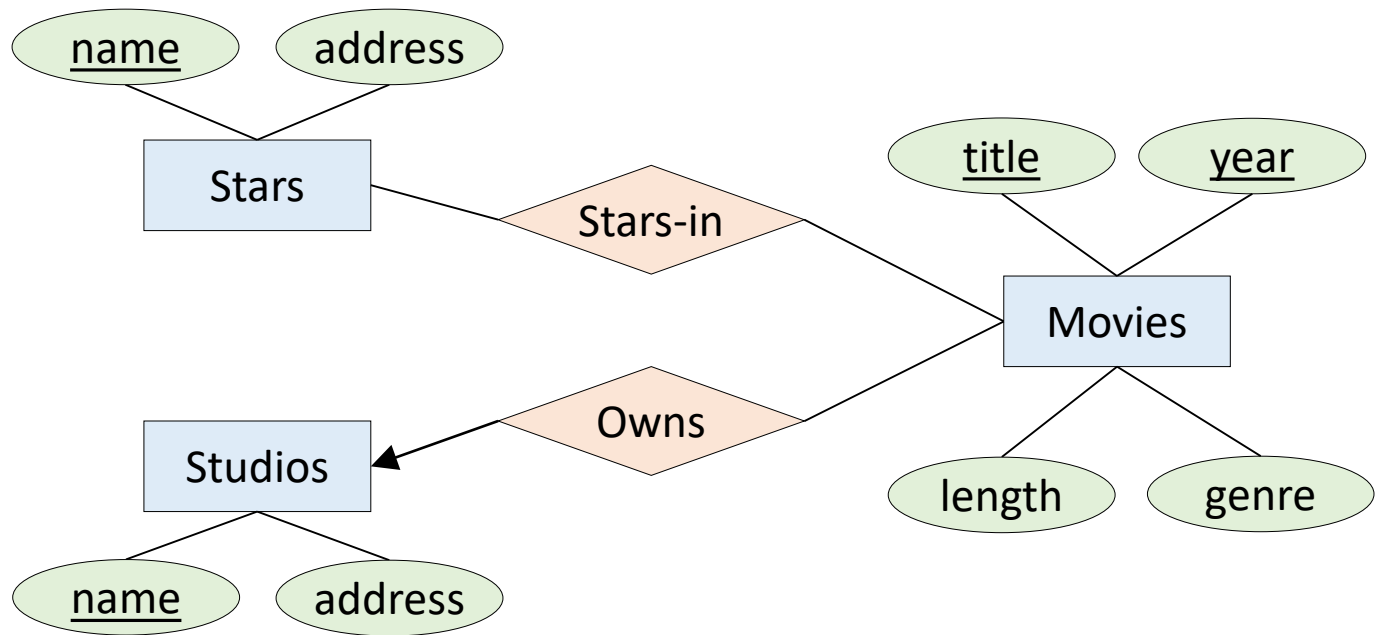
Movies

<u>title</u>	<u>year</u>	length	genre
Gone With the Wind	1939	231	drama
Star Wars	1977	124	sciFi
Wayne's World	1992	95	comedy

Stars

<u>name</u>	address
Carrie Fisher	123 Maple St., Hollywood
Mark Hamill	456 Oak Rd., Brentwood
Harrison Ford	789 Palm Dr., Beverly Hills

Example



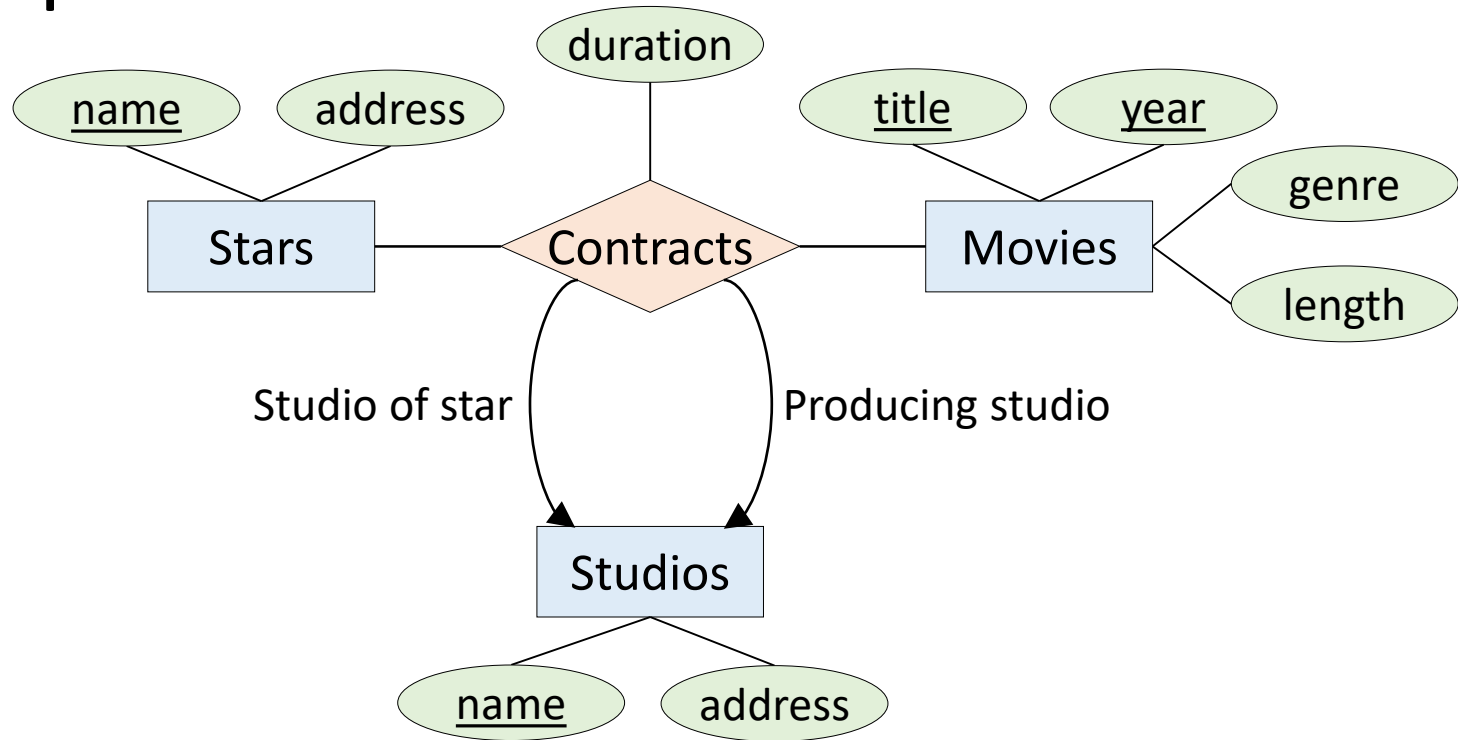
Stars-in

<u>title</u>	<u>year</u>	<u>starName</u>
Star Wars	1997	Carrie Fisher
Star Wars	1997	Mark Hamill
Star Wars	1997	Harrison Ford
Gone With the Wind	1939	Vivien Leigh
Wayne's World	1992	Dana Carvey
Wayne's World	1992	Mike Meyers

Owns

<u>title</u>	<u>year</u>	<u>studioName</u>
Star Wars	1997	Fox
Gone With the Wind	1939	MGM
Wayne's World	1992	Paramount

Example



Contracts

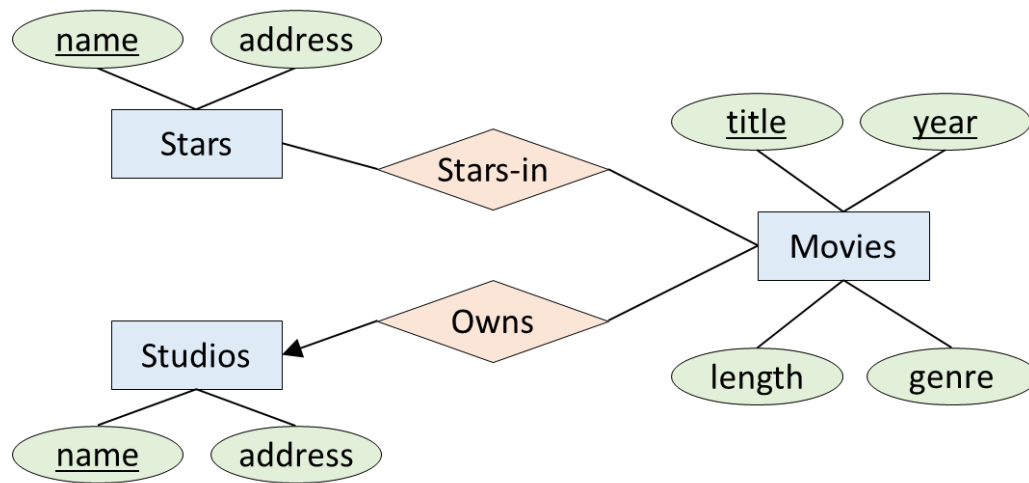
<u>starName</u>	<u>title</u>	<u>year</u>	studioOfStar	producingStudio	duration
-----------------	--------------	-------------	--------------	-----------------	----------

- **studioOfStar**: indicating the name of the studio for the role “Studio of star”
- **producingStudio**: indicating the name of the studio for the role “Producing studio”
- **duration**: attribute of the relationship set (not a part of the key)

Combining relations (Issue 1)

- Many-one relationship set R from entity set E to entity set F
 - all the attributes of the relation for R are functionally determined by the key for E
 - we can combine the relations for R and E into one relation for E
- Schema of the combined relation
 - ① all attributes of E
 - ② the key attributes of F
 - ③ any attributes belonging to R
- Note: an entity e of E that is not related to any entity of F
 - attributes of types (2) and (3) will have null values in the tuple for e

Example



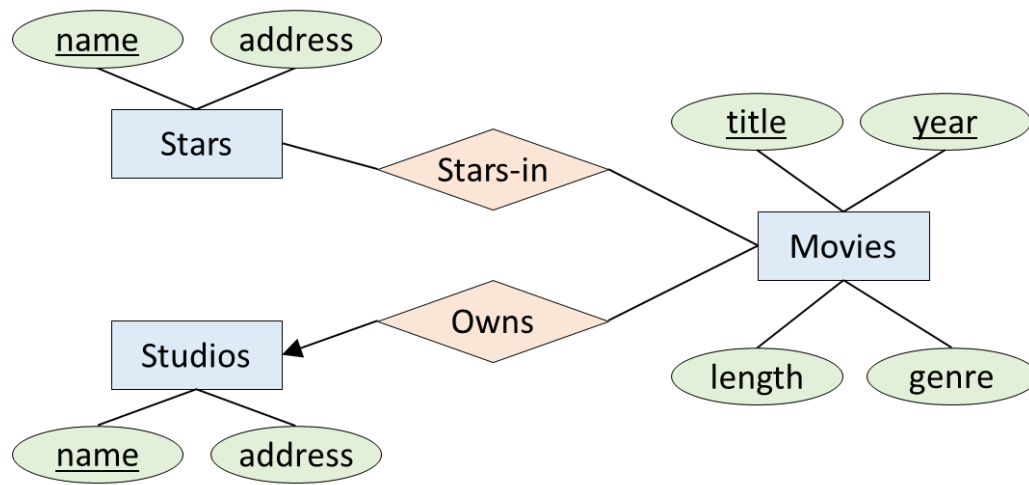
Movies	<u>title</u>	<u>year</u>	length	genre
	Gone With the Wind	1939	231	drama
	Star Wars	1977	124	sciFi
	Wayne's World	1992	95	comedy

Owns	<u>title</u>	<u>year</u>	studioName
	Star Wars	1997	Fox
	Gone With the Wind	1939	MGM
	Wayne's World	1992	Paramount

MovieOwns	<u>title</u>	<u>year</u>	length	genre	studioName
	Star Wars	1977	124	sciFi	Fox
	Gone With the Wind	1939	239	drama	MGM
	Wayne's World	1992	95	comedy	Paramount

Example

wrong combining
MovieOwns with Stars-in



MovieOwns

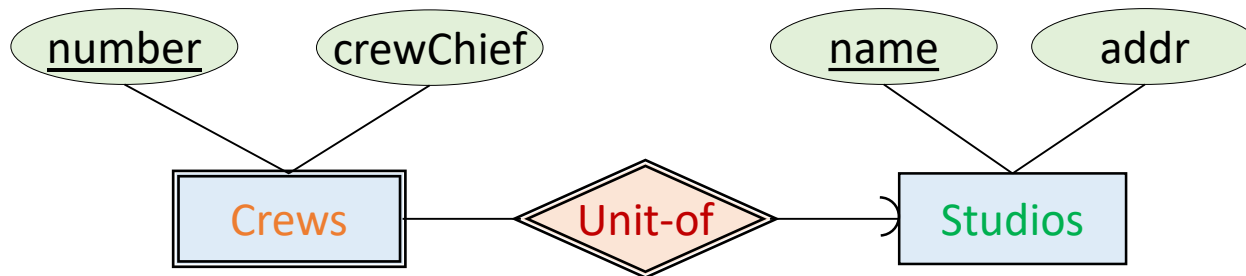
<u>title</u>	<u>year</u>	length	genre	studioName
Star Wars	1977	124	sciFi	Fox
Gone With the Wind	1939	239	drama	MGM
Wayne's World	1992	95	comedy	Paramount

MovieOwnsStarsin (has anomalies that we need to remove by normalization)

<u>title</u>	<u>year</u>	length	genre	studioName	<u>starName</u>
Star Wars	1977	124	sciFi	Fox	Carrie Fisher
Star Wars	1977	124	sciFi	Fox	Mark Hamill
Star Wars	1977	124	sciFi	Fox	Harrison Ford
Gone With the Wind	1939	239	drama	MGM	Vivien Leigh
Wayne's World	1992	95	comedy	Paramount	Dana Carvey
Wayne's World	1992	95	comedy	Paramount	Mike Meyers

Handling weak entity sets (Issue 2)

- When converting a weak entity set W to a relation
 - the relation must include **not only the attribute of W but also the key attributes of S** , the supporting entity set
 - the supporting relationship set **R need not be converted to a relation** at all
 - R is many-one relationship set from W to S , and so R 's attributes are combined into W
 - note: rename attributes, if necessary, to avoid name conflicts
- e.g.,



```
Studios(name, addr)
Crews(number, studioName, crewChief)
Unit-of(number, name)
```

Converting subclasses to relations (Issue 3)

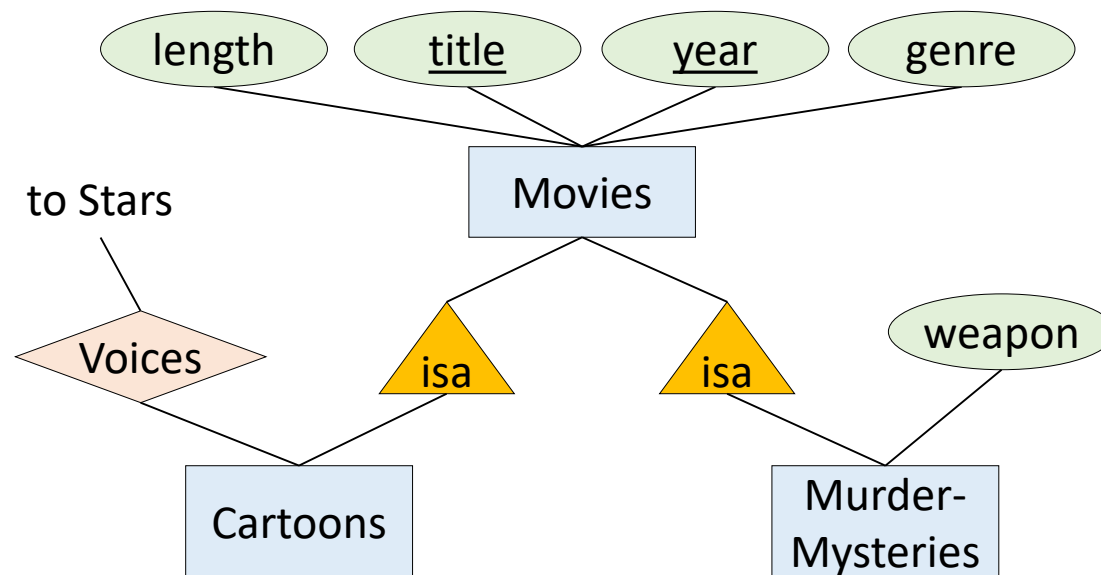
- Create a relation for each entity set E in the hierarchy
 - s.t., the relation for E includes the **key attributes of the root** and **any attributes belonging to E**
- Not create a relation for “isa” relationship set
 - unlike other relationship sets, it **connects components of a single entity** (**not connects distinct entities**)

Example: ER-style conversion

- Schemas

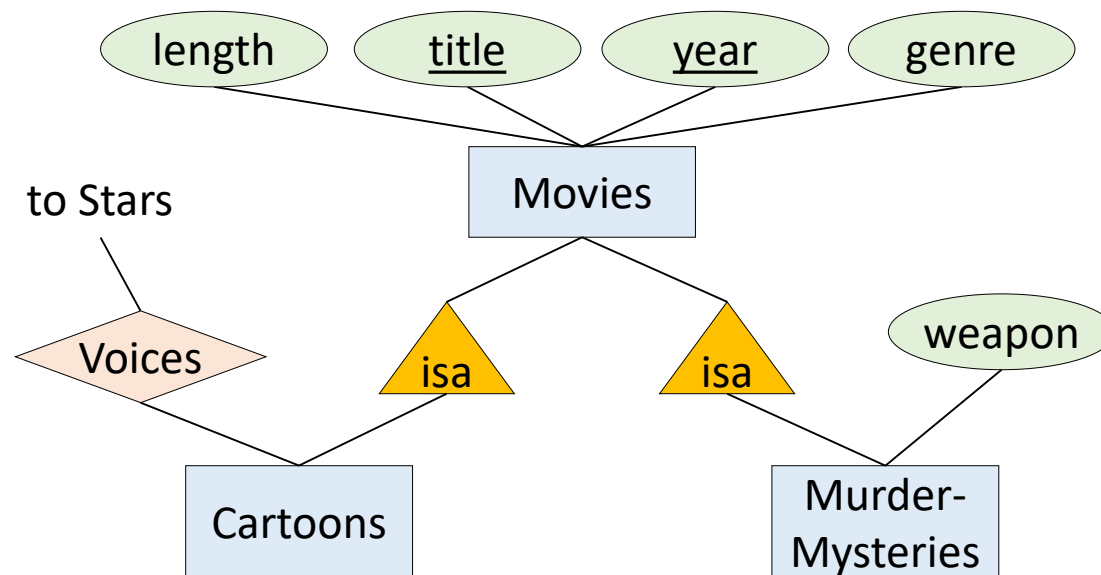
- `Movies(title, year, length, genre)`
- `MurderMysteries(title, year, weapon)`
- `Cartoons(title, year)`
- `Voices(title, year, starName)`

- Movies that are both cartoons and murder mysteries have tuples in all three relations



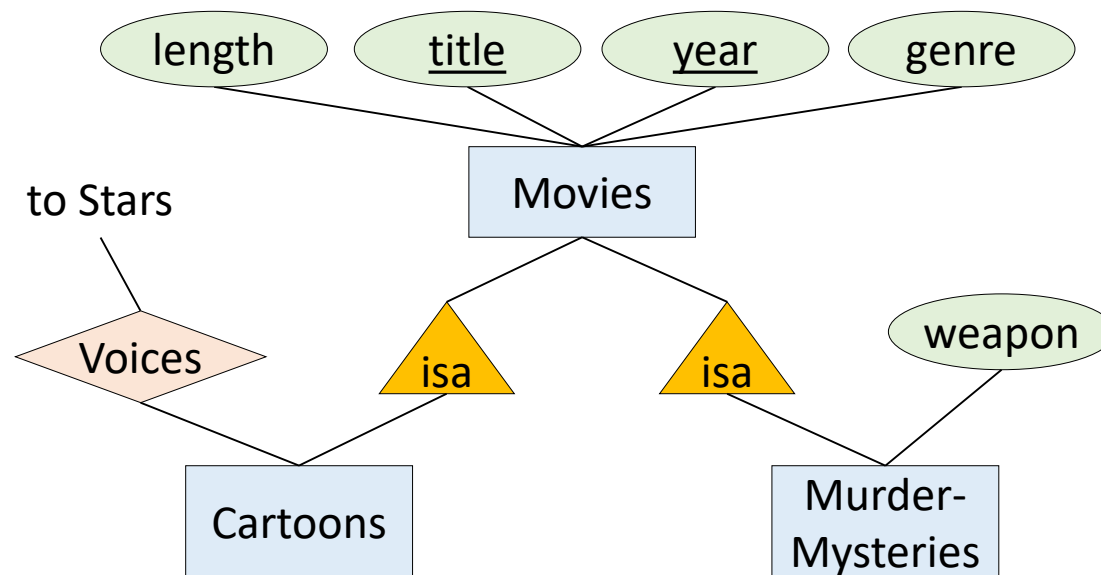
Example: an object-oriented approach

- Motivation: entities are objects that **belong to one and only one class**
 - Movies only: `Movies(title, year, length, genre)`
 - Movies and M-M only: `MoviesMM(title, year, length, genre, weapon)`
 - Movies and Cartoons only: `MoviesC(title, year, length, genre)`
 - All three entity sets: `MoviesCMM(title, year, length, genre, weapon)`
 - `Voices(title, year, starName)`



Example: using null values to combine relations

- Single relation that has all the attributes in the hierarchy
- Each tuple has **NULL** in the attributes that are not defined for that entity
- Schemas
 - Movies (title, year, length, genre, weapon)
 - Voices (title, year, starName)



Thank you

Q & A